Práctica 0: Puesta a punto con Python y OpenCV

Asignatura: Aplicaciones Avanzadas de Visión Artificial

Duración: 2 horas

Modalidad: Sesión práctica guiada

Objetivo general: Verificar el entorno de trabajo y repasar los fundamentos de Python, NumPy y OpenCV necesarios para las prácticas posteriores de la asignatura.

---

1. Objetivos de aprendizaje

Al finalizar la sesión, el estudiante deberá ser capaz de:

- Comprobar que su entorno de Python está correctamente configurado.
- Importar y utilizar NumPy, OpenCV y Matplotlib.
- Cargar, inspeccionar y guardar imágenes con OpenCV.
- Entender una imagen como un array de NumPy.
- Acceder y modificar píxeles, canales y regiones de interés.
- Aplicar transformaciones geométricas sencillas.
- Convertir entre espacios de color.
- Aplicar operaciones básicas de filtrado y segmentación.
- Construir un pequeño flujo de procesamiento de imagen sin interfaz gráfica.

> Esta práctica no pretende enseñar Python desde cero. Su finalidad es detectar carencias antes de comenzar con detección de objetos, segmentación, pose, identificación visual y análisis de vídeo.

---

2. Material necesario

- Python 3.10 o superior.
- NumPy.
- OpenCV para Python (opencv-python).
- Matplotlib.
- Jupyter Notebook, Google Colab o un entorno equivalente.
- Una o varias imágenes de prueba.

Comprobación mínima:

```
import sys
import numpy as np
import cv2
import matplotlib
import matplotlib.pyplot as plt

print("Python:", sys.version)
print("NumPy:", np.__version__)
print("OpenCV:", cv2.__version__)
print("Matplotlib:", matplotlib.__version__)
```

Ejercicio 0 — Comprobar el entorno

Tarea

Crear un script llamado check_env.py que:

1. Importe NumPy, OpenCV y Matplotlib.
2. Muestre la versión de Python.
3. Muestre la versión de cada biblioteca.
4. Cree un array de NumPy de tamaño 100 x 100.
5. Guarde ese array como una imagen utilizando OpenCV.

Ejemplo de array:

```
img = np.zeros((100, 100), dtype=np.uint8)
```

Guardar con:

```
saved = cv2.imwrite("test.png", img)
print("Imagen guardada:", saved)
```

Comprobación

Al finalizar debe existir un archivo test.png válido.

Pregunta

¿Qué tipo de dato utiliza NumPy para representar los valores de la imagen?

Ejercicio 1 — Cargar e inspeccionar una imagen

Tarea

Cargar una imagen utilizando:

```
img = cv2.imread("imagen.jpg")
if img is None: raise FileNotFoundError("No se pudo cargar la imagen")
```

Mostrar:

```
print("Tipo:", type(img))
print("Dimensiones:", img.shape)
print("Tipo de dato:", img.dtype)
print("Valor mínimo:", img.min())
print("Valor máximo:", img.max())
```

Responder:

1. ¿Qué tipo de objeto devuelve cv2.imread()?
2. ¿Qué representan las dimensiones de shape?
3. ¿Cuál es el rango habitual de valores de una imagen de 8 bits?
4. ¿Qué ocurre si el archivo no existe?

Ejercicio 2 — La imagen como array de NumPy

Tarea A — Acceso a píxeles

Seleccionar un píxel:

```
height, width = img.shape[:2]

y, x = height // 2, width // 2
pixel = img[y, x]
print(pixel)
```

Modificar una pequeña región rectangular de la imagen.

Ejemplo:

```
y1, y2 = height // 4, 3 * height // 4
x1, x2 = width // 4, 3 * width // 4
modified = img.copy()
modified[y1:y2, x1:x2] = 0
cv2.imwrite("region_modificada.png", modified)
```

Guardar el resultado.

Tarea B — Región de interés

Extraer una región:

```
roi = img[y1:y2, x1:x2]
cv2.imwrite("roi.png", roi)
```

Guardar únicamente esa región.

Tarea C — Copias y vistas

Comparar:

```
roi_view = img[y1:y2, x1:x2]
roi_copy = img[y1:y2, x1:x2].copy()
roi_view[:] = 0
roi_copy[:] = 255
```

Comprobar que modificar roi_view también modifica img.

Comprobar que modificar roi_copy no altera img.

Pregunta

¿Por qué es importante distinguir entre una vista y una copia al trabajar con imágenes grandes?

Ejercicio 3 — Canales y espacios de color

OpenCV utiliza por defecto el orden:

```
BGR
```

en lugar de:

```
RGB
```

Tarea A — Separar canales

```
b, g, r = cv2.split(img)
cv2.imwrite("canal_b.png", b)
cv2.imwrite("canal_g.png", g)
cv2.imwrite("canal_r.png", r)
```

Guardar cada canal como una imagen independiente.

Tarea B — Conversión a RGB

```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

plt.imshow(rgb)
plt.axis("off")
plt.show()

Tarea C — Escala de grises

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imwrite("01_gray.png", gray)
```

Comparar:

```
print("Dimensiones BGR:", img.shape)
print("Dimensiones en gris:", gray.shape)
```

Pregunta

¿Por qué una imagen en escala de grises tiene dos dimensiones mientras que una imagen BGR tiene tres?

Ejercicio 4 — Transformaciones geométricas

Tarea A — Redimensionar

Reducir la imagen al 50 % de su tamaño.

```
small = cv2.resize(
   img,
   None,
   fx=0.5,
   fy=0.5,
   interpolation=cv2.INTER_AREA
)
cv2.imwrite("small.png", small)
```

Tarea B — Volteo

Crear:

- una versión invertida horizontalmente;
- una versión invertida verticalmente.

Consultar el comportamiento de:

```
horizontal = cv2.flip(img, 1)
vertical = cv2.flip(img, 0)
cv2.imwrite("flip_horizontal.png", horizontal)
cv2.imwrite("flip_vertical.png", vertical)
```

Tarea C — Rotación

Rotar la imagen 90 grados.

Probar:

```
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
cv2.imwrite("rotated_90.png", rotated)
```

Pregunta

¿Por qué el tipo de interpolación puede ser importante al redimensionar imágenes que posteriormente serán procesadas por un modelo de visión?

Ejercicio 5 — Filtrado y bordes

Trabajar inicialmente sobre una imagen en escala de grises.

Tarea A — Suavizado

Aplicar un filtro gaussiano:

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
cv2.imwrite("02_blur.png", blur)
```

Repetir con los siguientes tamaños de núcleo:

```
3x3
5x5
11x11
21x21
```

Comparar visualmente.

Tarea B — Bordes

Aplicar Canny:

```
edges = cv2.Canny(gray, 100, 200)
cv2.imwrite("03_edges.png", edges)
```

Modificar los dos umbrales.

Responder:

- ¿Qué ocurre cuando disminuyen?
- ¿Qué ocurre cuando aumentan?

Tarea C — Flujo de procesamiento

Comparar:

```
gray → Canny
```

con:

```
gray → GaussianBlur → Canny
```

Pregunta

¿Por qué puede ser útil suavizar una imagen antes de calcular sus bordes?

Ejercicio 6 — Segmentación sencilla

Tarea A — Umbral fijo

Aplicar:

```
_, mask = cv2.threshold(
   gray,
   128,
   255,
   cv2.THRESH_BINARY
)
cv2.imwrite("04_mask_threshold.png", mask)
```

Probar diferentes valores de umbral.

Tarea B — Otsu

Aplicar:

```
threshold, mask_otsu = cv2.threshold(
   gray,
   0,
   255,
   cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Mostrar el valor de umbral calculado automáticamente:

```
print("Umbral de Otsu:", threshold)
cv2.imwrite("04_mask_otsu.png", mask_otsu)
```

Comparación

Comparar visualmente:

1. imagen original;
2. escala de grises;
3. umbral fijo;
4. Otsu.

Pregunta

¿En qué situaciones un único umbral global puede ser insuficiente?

Ejercicio 7 — Mini-reto final

Construir un programa llamado:

```
pipeline.py
```

que reciba una imagen y genere automáticamente los siguientes archivos:

```
01_gray.png
02_blur.png
03_edges.png
04_mask.png
```

Flujo de procesamiento requerido:

```
Imagen → Escala de grises → Filtro gaussiano
Rama 1: Filtro gaussiano → Canny → 03_edges.png
Rama 2: Filtro gaussiano → Threshold / Otsu → 04_mask.png
```

El programa debe imprimir además:

```
Resolución original
Tipo de dato
Número de canales
Valor medio de intensidad
Desviación típica
```

Los dos últimos valores pueden obtenerse con NumPy:

```
mean_intensity = np.mean(gray)
print("Intensidad media:", mean_intensity)
std_intensity = np.std(gray)
print("Desviación típica:", std_intensity)
```

Resultado esperado de la práctica

Al finalizar la sesión, cada estudiante debería disponer de:

```
P0/
├── check_env.py
├── pipeline.py
├── input/
│   └── imagen.jpg
└── output/
    ├── 01_gray.png
    ├── 02_blur.png
    ├── 03_edges.png
    └── 04_mask.png
```

Cuestiones de autoevaluación

Antes de comenzar las siguientes prácticas, el estudiante debería poder responder afirmativamente a las siguientes preguntas:

- ¿Sé cargar y guardar imágenes con OpenCV?
- ¿Entiendo qué representa `shape` en una imagen?
- ¿Sé diferenciar BGR, RGB y escala de grises?
- ¿Sé seleccionar una región utilizando indexación por rangos de NumPy?
- ¿Entiendo la diferencia entre una copia y una vista?
- ¿Sé redimensionar una imagen?
- ¿Sé aplicar un filtro básico?
- ¿Sé obtener una imagen de bordes?
- ¿Sé generar una máscara binaria?
- ¿Puedo construir un flujo de procesamiento sencillo sin utilizar una interfaz gráfica?

Si alguna respuesta es negativa, conviene revisar ese contenido antes de comenzar las prácticas avanzadas.

Recursos de consulta

- OpenCV-Python Tutorials: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- NumPy Documentation: https://numpy.org/doc/stable/
- Python Tutorial: https://docs.python.org/3/tutorial/
- Stanford CS231n — Python/NumPy Tutorial: https://cs231n.github.io/python-numpy-tutorial/